

Enhancing Fault Tolerance in High-Performance Computing: A Real Hardware Case Study on a RISC-V Vector Processing Unit

MARCELLO BARBIROTTA ¹, FRANCESCO MINERVINI ², CARLOS ROJAS MORALES ², ADRIAN CRISTAL ²,
OSMAN UNSAL ², AND MAURO OLIVIERI ¹ (Senior Member, IEEE)

¹Department of Information Engineering, Electronics and Telecommunications, Sapienza University of Rome, Via Eudossiana 00184, Italy

²Barcelona Supercomputing Center, Barcelona 08034, Spain

CORRESPONDING AUTHOR: MARCELLO BARBIROTTA (e-mail: marcello.barbirotta@uniroma1.it)

ABSTRACT High-Performance Computing (HPC) systems are designed for large-scale processing and complex dataset analysis leveraging scalability, efficiency, and parallelism, often integrating specialized hardware structures such as Vector Processing Units (VPUs). As these systems have grown in complexity and scale, their vulnerability to errors and failures has become an important and complex issue in the HPC world. Our research addresses this challenge by exploring and implementing advanced fault tolerance techniques inside the Vitruvius+ architecture, a partial out-of-order Vector Processing Unit. To the best of our knowledge, this is the first full RTL-level implementation of instruction replication in an HPC-class vector processor for reliability. Specifically, we investigate the integration and interaction of redundancy mechanisms inside the most sensitive architectural units, obtaining a reduction of 75% in non-silent faults causing system failure, proven by an extensive fault injection simulation campaign, with a hardware overhead of only 7.5% and a negligible variation in clock frequency.

INDEX TERMS Fault injection, fault tolerance, high-performance computing, RISC-V, vector processing unit.

I. INTRODUCTION

Fault Tolerance (FT) techniques were first applied to computational structures with the evolution of the space age, due to the faults caused by cosmic rays, and subsequently moved to the industrial market addressing safety critical applications [1]. In High-Performance Computing (HPC) systems, the application program is parallelized on many thousands of computing nodes, each having tens of processing cores. If a generic processing core has a Mean Time Between Failures (MTBF) of 100 years, a machine with 1000000 cores may experience a failure every 53 minutes on average [2]. As a result, FT has become a primary issue in the design of today's and future HPC systems.

Nonetheless, due to the high hardware complexity and the stringent performance requirements, today's HPC systems are not equipped with extensive FT hardware mechanisms. FT

techniques are implemented at the software level, and/or possibly relying on Error Correcting Code (ECC) support in the memory subsystems.

In the literature, the main standard for FT within HPC systems is represented by software checkpointing with rollback recovery methodologies [2]. Efficient checkpointing/recovery is complex due to its close connection with key components of the HPC system, such as libraries, programming models, and data storage mechanisms [3]. It has been documented that pure checkpointing/recovery techniques at large scale may result in severe performance drop [4], with up to 6X - 8X overhead in latency [5]. For this reason, process or thread replication at the software level has been employed, so that a failure in one processing element does not compromise the overall application. Despite the apparently wasteful nature of replication, it can be more efficient than an aggressive use of

checkpointing, in peta-scale and exa-scale HPC systems [2], however software-level replication is still costly in terms of the overhead it incurs. What is needed is a hardware driven mechanism, thus minimizing the overhead.

This work is the first to explore the potential grade of FT offered by hardware support for replication at the instruction level inside an HPC system processing core, rather than a software implementation of replication at the process or thread level. In particular, we address the class of processing cores commonly referred to as vector processors [6].

Vector processors are gaining momentum from industrial and academic perspectives due to their intrinsic efficient exploitation of Data Level Parallelism (DLP) on large datasets typical of Machine Learning, AI, and Big Data applications, achieving high performance while consuming relatively less power compared to other types of processors [7]. Instruction replication is particularly relevant in this context as it protects the vector processor pipeline, floating-point units, and integer ALUs, which dominate the vector processor area and are thus more vulnerable to soft errors. The technique is complementary to existing FT approaches based on process/thread level checkpointing, aiming at reducing the occurrence of failures to be recovered by a system rollback procedure.

Our work draws up the modifications adopted to implement hardware FT in the computing node developed by a consortium of European companies and universities within the European Processor Initiative (EPI) research program [8]. In particular, we modified the vector processing acceleration sub-system of the computing node, called Vitruvius+ Vector Processing Unit (VPU) [9], which is in charge of executing the vector instructions. The adopted approach for implementing FT hardware support in the VPU relies on the concepts of instruction level replication, instruction level roll-back, and ECC protection. In particular, replication is obtained by a novel Hardware Instruction Duplication technique within the decoding stages of the VPU microarchitecture, resulting in temporal redundancy, capable of detecting faults before the instruction is committed. In case of detection, vector instruction roll-back is implemented by adapting a kill-and-retry hardware mechanism of the VPU control subsystem, exploiting register renaming. An ECC protection sub-system is applied to the Vector Register File to detect faults occurring on data produced by already committed instructions.

The contributions of the proposed work are the following:

- we demonstrate the hardware implementation of the proposed Hardware instruction replication in a vector microarchitecture, down to synthesizable Register Transfer Level (RTL) design;
- we set up an ad-hoc Fault Injection (FI) simulation environment for the validation of the design and integrated within a standard Universal Verification Methodology (UVM) environment;
- we quantitatively demonstrate an 85% reduction in the statistical occurrence of computing node failures, by analysing the FI results, with little hardware overhead.

The rest of the article is organized as follows: Section II discusses the related works about FT in high-performance

processors; Section III describes the Vitruvius+ microarchitecture highlighting the features relevant to the proposed design; Section IV discusses the fault tolerant VPU with the applied hardware microarchitecture modifications; Section V analyzes the hardware overhead and the results of FI simulation in the UVM-based environment. Finally, Section VI summarizes the conclusions.

II. RELATED WORKS

In order to mitigate transient errors in logic circuits, one circuit-level approach is radiation-hardening which applies transistor sizing at design time; this approach (robustness by technology) is the preferred one for space applications however it has a prohibitively high area cost for HPC chip designs which need to have high compute density per square millimeter. Another approach is to implement special hardware/software architecture features, which tolerate device errors and depend on detecting them thanks to spatial or temporal redundancy (robustness by design). This is the approach taken by mainstream designs for data center, desktop and up to a degree for HPC.

The renewed interest in vector processors for HPC and AI has been signified by the advent of various ISA extensions like Intel AVX-512 [10], ARM SVE [11] and RISC-V RVV [12], as well as vector-enabled supercomputers like NEC SX Architecture [13] and Fugaku [14]. Operations in vector processors are executed on arrays of data, called vector registers, composed of vector elements. The Maximum Vector Length (MVL), which determines the maximum number of elements of each vector register, as well as the number of vector registers are defined by a dedicated Instruction Set Architecture (ISA). In the physical implementation, the vector registers compose the Vector Register File (VRF), and a dedicated interconnect allows data movement between the functional units and the vector registers [6]. Besides the most common operations such as element-wise integer and floating-point arithmetic operations between vectors, vector processors implement a set of specific operations. In particular, vector reductions apply certain arithmetic operations to reduce a whole vector register to a single scalar result. Mixed-precision operations are also supported, operating on vector registers whose elements are manipulated using different precision. In addition, vector processors also implement particular memory operations, like *strided* loads and stores (accessing vector elements in memory at locations calculated with a constant offset, called stride, from a base address) and gather-scatter operations (accessing vector elements in memory at locations calculated with a set of indices, which are elements of another vector, to be computed with a base address) [9].

The general checkpointing technique in HPC systems is implemented in two main approaches: coordinated/uncoordinated checkpointing [15] and hierarchical checkpointing [16]. In the former approach, part of the execution is lost when an error occurs, and the checkpointing protocol restores the application to a past stage by recalculating the entire state of the processor (in the coordinated case) or by

recalculating only the state of processes that have been subject to failure (in the un-coordinated case). The hierarchical checkpointing approach allows combining both techniques by acting specifically on groups of similar processes, carrying out coordinated recovery only on those processes or uncoordinated recovery between different groups of processes. The operation of checkpointing is a process composed by parallel threads that may be implemented in the operating system [17], in library functions, inserted by the compiler [18], or created at the user level [19].

In all cases, state saving must be accomplished, which may run asynchronously and in parallel with the continuous progress of the process, each time some tasks are reached (like in the parallelizer approaches [20]) or each time a group of tasks are executed and verified (like execution and verify approaches [21]). The generated checkpoint can be saved in various ways, considering the available space, in protected memory regions near or far from the original region, or in multiple spatially separated points [16].

In memory subsystems, the established countermeasure for stored data corruption is the use of Error Correcting Codes (ECC), which are available in the caches of many high-end processors [22], [23]. The occurrence of faults in the data-path and control-path of the computing nodes are generally mitigated by means of redundant computation, either in the form of spatial redundancy or in the form of temporal redundancy.

In spatial redundancy approaches, the same computation is executed on multiple hardware resources, possibly implementing a Double Modular Redundancy (DMR) or Triple Modular Redundancy (TMR) [24]. In temporal redundancy approaches, the task execution is replicated in time [25] on the same hardware resource. In the context of highly parallel HPC workloads, if we assume a fixed amount of available hardware resources both approaches come at the cost of increased execution time, since spatial redundancy sacrifices part of the parallelism of the machine.

Temporal redundancy is in fact, widely used in different embedded platforms, including multi-core and multi-threaded architectures [26], [27], [28], [29] where the aspect of redundant multithreading at application-level holds notable significance, as highlighted in [30], [31]. Regarding HPC systems, some effort has been done to achieve temporal redundancy via software replication. Authors in [32] have proposed an HPC node based on an ARM multi-core processor with redundancy at the software level, along with a software monitor to detect errors, trading-off on increasing and decreasing the monitoring frequency due to the performance overhead of the monitor.

Hyman et al. [33] have proposed a hardware/software solution targeting instruction level redundancy in general-purpose multi-core high performance processors, implementing a mix of techniques including temporal and spatial redundancy. Thanks to the inherent redundancy of resources available in multiple issue processors, a compiler-driven instruction replication is activated on the same core whenever the instruction dependency allows it, or otherwise on another core

via a dedicated arbitrating interconnect. Also, the approach includes redundancy in arithmetic operations at the functional unit level. Upon detection of a fault, the correct operation is recovered by means of a hardware supported roll-back relying on a specialized reorder buffer. Overall, instruction level simulations show that the approach can achieve a very low execution time overhead, depending on the characteristics of the software workload. The hardware overhead is estimated as much as 5% for the arbitrating interconnect only, while no synthesizable implementation is reported of the core microarchitectures, which include several additional logic blocks to support replication, detection, and roll-back. Regarding the achieved resilience, the work conjectures a 100% fault coverage, yet not supported by any bit-level fault injection simulation, due to the lack of an RTL implementation of the design.

Other works have addressed selective or partial redundancy mitigation schemes, protecting only essential system parts in order to reduce the overhead sacrificing the fault coverage. The work in [34] presents the Self-Imposed Temporal Redundancy (SITR) technique, applied only to functional units such as adders, multipliers, and shifters. The same functional unit executes the calculation twice in two consecutive cycles. To limit the performance loss, the value produced by the first calculation is propagated and then flushed only if voting with the second calculation detects an error. Of course, this does not prevent the functional unit busy time to be doubled. Authors in [25] address task-level replication, approximating the task before it is replicated, thus combining approximate computing techniques with temporal redundancy. The execution time overhead is reduced thanks to the approximate version of the original task being re-executed.

All of the above techniques, along with more conventional pure software checkpointing/roll-back techniques, address general purpose computation. To the best of our knowledge, no technique tailored to HPC vector computing has been reported in the literature to date. A hardware redundancy approach for vector acceleration in embedded edge-computing is described in [35], targeting an accelerator that supports a custom set of basic vector operation extending the RISC-V instruction set. As opposed to this, the proposed work addresses a Vector Processing Unit (VPU) supporting the full RISC-V “V” vector instruction set extension, along with superscalar execution, integrated in a real HPC computing node.

The approach proposed in this work is based on temporal redundancy in conjunction with ECC, specifically tailored to the VPU microarchitecture. It complements both conventional software-based checkpointing/roll-back techniques and the more aggressive hardware/software techniques on general purpose multi-cores. The primary goal of the proposed design is to give the possibility of running critical parts of HPC vectorized computation in a fault-tolerant mode, sacrificing part of the speed to dramatically reduce the occurrence of failures that would require a roll-back of the system. For this reason, the main objective of the proposed design is to give evidence of the fault coverage achieved by the proposed technique by

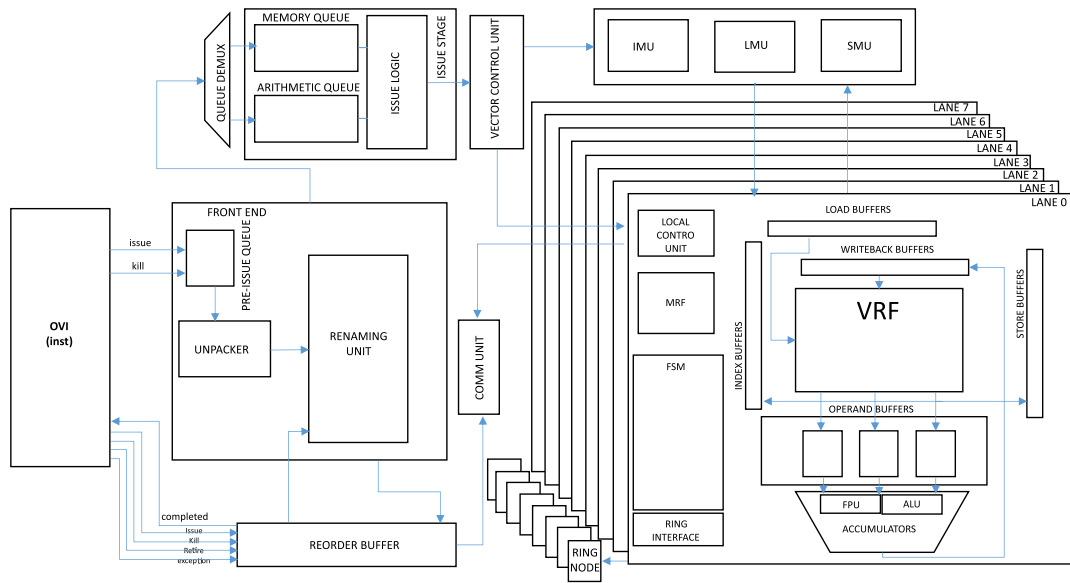


FIGURE 1. Vitruvius+ Architecture.

means of an extensive FI simulation on a fully synthesizable RTL implementation.

III. VECTOR PROCESSING UNIT

The Vitruvius+ Architecture [9] is a decoupled vector accelerator with lightweight out-of-order execution capabilities, boosted by vector register renaming and concurrent execution of memory and arithmetic instructions. Vitruvius+ mainly targets HPC applications characterized by long vectors. The first version of Vitruvius+ was compliant with the RISC-V Vector (RVV) extension version 0.7.1, while in its latest implementation it has been adapted to be compliant with RVV version 1.0 [36].

A. GENERAL ARCHITECTURE FEATURES

By natively supporting long vectors, Vitruvius+ is well suited to support applications with high DLP. Referring to the RISC-V standard, Vitruvius+ implements the following architectural parameters:

- *ELEN*, the maximum size in bit of a single vector element, is set to 64 bits. As such, Vitruvius+ supports element widths of 8, 16, 32, and 64 bits.
- *VLEN*, where $VLEN \geq ELEN$, the number of bits in a vector register, is set to 16384 bits. This means that Vitruvius+ is capable of managing any vector length up to 256 Double-Precision (DP) 64-bit floating-point elements.

Vitruvius+ is a highly parametrized design. In fact, the maximum vector length, which implicitly affects the size of the VRF, as well as the number of functional units, can be set at synthesis time. Vitruvius+ internally splits the VRF and the functional units into parallel execution pipeline, usually referred to as lanes. The baseline architecture with 16384-bit vectors instantiates 8 parallel vector lanes, each lane equipped

with one Arithmetic Logic Unit (ALU) and one Floating-point Unit (FPU). Each lane has its own control Finite State Machine (FSM), and includes a slice of the VRF. In this implementation, each slice of the VRF accounts for 10 KB of SRAM memory, organized as five 2-KB single-port banks. As such, the overall Vitruvius+ VRF amounts to 80 KB, representing the most area-consuming block of the VPU. Fig. 1 shows the high-level block diagram of the Vitruvius+ vector accelerator. The main components of the vector accelerator are briefly described below.

B. SCALAR CORE INTERFACE

Vitruvius+ is compliant with the Open Vector Interface (OVI) [37]. The OVI is an open-source interface specification of the communication between a scalar core and a vector unit through a connection composed of specific groups of signals. While a more detailed description can be found in the literature, here we summarize the main functionalities of all the groups:

- *ISSUE*: it is used to send instructions from the scalar core to the VPU, together with the specific instruction identifier and useful information such as a possible scalar operand and the current vector Control and Status Register (CSR) configuration.
- *DISPATCH*: it is used to inform the VPU that certain instructions can commit or have to be killed, in this case enabling a roll-back of the state of the vector unit. This feature is particularly relevant in the view of the fault tolerant design of the VPU.
- *COMPLETED*: it is used by the VPU to communicate the completion of the instructions and to possibly propagate any scalar results to the scalar core.
- *MEMOP*: it triggers the start and the end of any memory operation. In OVI, the scalar core takes care of the vector

memory operations, with the VPU handling the transfers of indices and data to and from the scalar core.

- **LOAD:** on this group of signals, the scalar core provides the data coming from the memory as a result of a vector load operation, together with metadata for Vitruvius+ to understand which vector elements the data belong to.
- **STORE:** on this group of signals, the VPU provides the data for a vector store operation, in order to transfer them to the memory via the scalar core.
- **MASK_IDX:** the VPU uses this bus to transfer both indices and masks to the scalar core, when a masked or indexed memory operation is being executed.

Except for the interaction between the scalar core and the vector unit for the execution of memory instructions, the OVI standard allows for decoupling the scalar execution stream from the vector execution stream. As such, Vitruvius+ can be connected to any scalar core which supports the OVI specifications.

C. VECTOR REORDER BUFFER (VROB) UNIT

As the execution of arithmetic and memory instructions can be completed out-of-order, a reorder buffer is used to keep the instruction commit order. Whenever an instruction completes the execution, the *commit unit* notifies the *vector reorder buffer (VROB)*. The VROB correspondingly drives the signals of the OVI interface to notify the scalar core, which replies with a “*retire_valid*” signal. At this point the VROB activates the instruction commit process of the instruction and the head of the VROB moves to the next instruction in dispatch order. The VROB also receives the signals to possibly trigger a roll-back of the VPU state, in case a kill command is issued by the scalar core via the OVI or in case an exception raises. When an instruction has been committed, it becomes not possible any more to retrieve the state of the VPU as before the instruction was executed.

D. VECTOR INSTRUCTION PIPELINE

The Vitruvius+ front-end is the first stage traversed by any new incoming instruction. It is composed of three main blocks:

- The *pre-issue queue* collects the instructions as they come from the OVI interface and stores them in an internal First In First Out (FIFO) queue, in preparation towards the subsequent stages.
- The *unpacker* decodes the instruction and generates microoperations for the execution of each instruction.
- The *renaming unit* is the key enabler of out of order execution in the VPU. The unit eliminates the WAW and WAR dependencies, by maintaining a Register Alias Table (RAT), which maps each logical register to the last assigned physical register, and a Free Register List (FRL), which keeps track of the available physical registers. The Vitruvius+ designs supports up to 40 physical vector registers in the VRF, as reported in [9].

After leaving the front-end stage, the vector instructions are classified by the *queue demux* which detects to which

of the issue queues the instruction has to be sent. At this point, the instructions are in the issue stage and wait for the *vector control unit* to allow for the start of the execution. This mechanism, coupled with vector register renaming, allows the concurrent execution of memory and arithmetic instructions. Whenever a new memory instruction begins execution, one of the memory units between the *Index Management Unit (IMU)*, the *Load Management Unit (LMU)*, and the *Store Management Unit (SMU)* is enabled, for transferring indices and masks, receiving load data, or transferring store data, respectively. Each *lane* is also notified about the start of the memory operations, and prepares the corresponding buffers to send or receive data from the memory units. Each lane instantiates one slice of the VRF, a local FSM, a slice of the *Mask Register File (MRF)*, one FPU, one ALU, along with the dedicated logic for reductions and a set of buffers for temporarily storing the data that is read from or is going to be written to the VRF. Data movement among the lanes is possible through a low power area-efficient ring interconnect.

IV. FAULT TOLERANT VECTOR PROCESSING UNIT

The Fault Tolerant Vitruvius+ Architecture, depicted in Fig. 2, maintains the same vector characteristics in the original version.

Generally speaking, temporal redundancy based on result comparison between replicated instructions would lead to complex and slow logic within a VPU, capable of producing extremely long vectors with double-precision floating-point elements. In the proposed design we take advantage of the ECC calculation, already present for protecting the VRF, to compare the calculated ECC values of each replicated instruction results, before writing the value in the VRF, rather than comparing the whole replicated result vectors. The approach is implemented by the additional dedicated hardware units described below. The approach is implemented by the additional dedicated hardware units described below, and can be implemented in any decoupled vector processor operating in a multi-lane, out-of-order scheme. Instruction duplication can be implemented at the scalar/vector interface when the scalar core dispatches the vector instructions, while ECC addition can be implemented in the vector register file read/write logic.

A. ERROR CORRECTING CODE LOGIC

The red blocks in Fig. 2 implement ECC support inside the VRF. This logic implements a protection mechanism inside the VRF featuring Single Error Correction Double Error Detection (SEC-DED). Moreover, it implements the ECC comparisons inside the ECC buffers in conjunction with the hardware instruction duplication features (see Section IV-B5). Having 64-bit of double precision data, according to the Hamming rules it is possible to add 8-bit parity redundancy for every data word, incrementing each register bank by 12.5%, thus increasing the total VRF memory size from 80 KB up to 90 KB.

A problem related to implementing an ECC mechanism within such a large VRF lies in the performance degradation

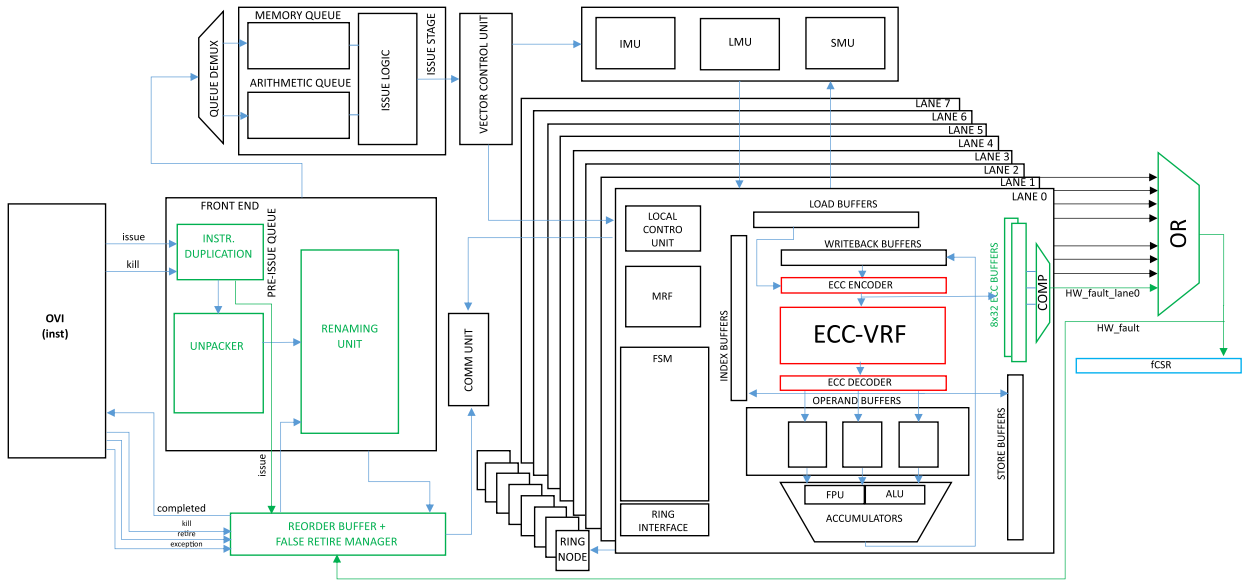


FIGURE 2. Fault tolerant - Vitruvius+ Architecture.

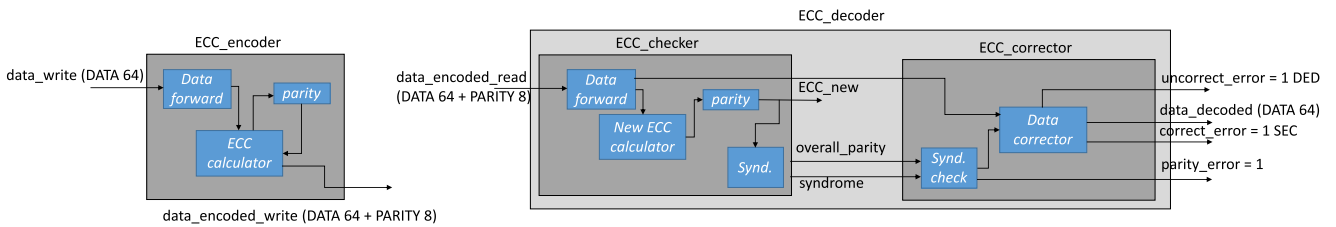


FIGURE 3. VRF encoder and decoder mechanism.

associated to the latency of error correction and rewriting of the corrected data in the VRF. This negative impact can be mitigated if the rewriting operation is postponed, yet the corrected data is immediately used in the upcoming instructions. To facilitate this, we implemented the Encoding and Decoding mechanisms directly in read and write access logic of the VRF as follows (Fig. 3):

- Encoder: The 64-bit word coming from the Write-back Buffers or the Load Buffers is assigned to a specific slice (lane) of the VRF based on a shuffling system that follows the data organization of Vitruvius+, and is then locally encoded within the Encoder block within the lane. The Encoder block calculates the ECC with parity, producing a 72-bit encoded data word.
- Decoder: The Decoder block within each lane receives the 72-bit data word read from the VRF and it calculates the new ECC and the Syndrome to detect possible errors. The Decoder block returns the decoded 64-bit data, correcting possible single errors or notifying multiple errors, notifying the following conditions:
 - Corrected Error (CE), in case of Single Error Correction (SEC).
 - Detected Double Error (DDE), in case of Double Error Detection (DED).
 - Detected Parity Error (DPE), in case the fault hits one of the 8-parity bits.

As mentioned above, in the case of SEC, the corrected data is immediately used for the upcoming operations from the bypass logic, but is not immediately rewritten inside the VRF. To perform a VRF correction, the signals produced by the Decoder block are counted in a custom Control and Status Register (CSR) (namely *fault-CSR*), accumulating the number of errors and accessible to the Operating System (OS) software running on the scalar core. When the count reaches a system-defined threshold, the OS can activate a software routine through a software interrupt to recover the faulty data in the VRF by performing a reading-writing cycle, similarly to a memory scrubbing mechanism. Overall, every single lane can perform encoding/decoding operations in the VRF banks assigned to it, as depicted in Fig. 2, counting errors inside the *fault-CSR* status register.

B. INSTRUCTION DUPLICATION

The goal of instruction replication is to produce multiple copies of the same operation in the compute pipeline in a time-staggered manner, i.e. employing temporal redundancy. The main advantage of these techniques is related to the almost null hardware overhead at the expense of time overhead. The green blocks in Fig. 2 are related to the temporal redundancy hardware support, which implements the Instruction Duplication for all the arithmetic instructions within the Front End and the Reorder Buffer blocks. Every duplicated instruction

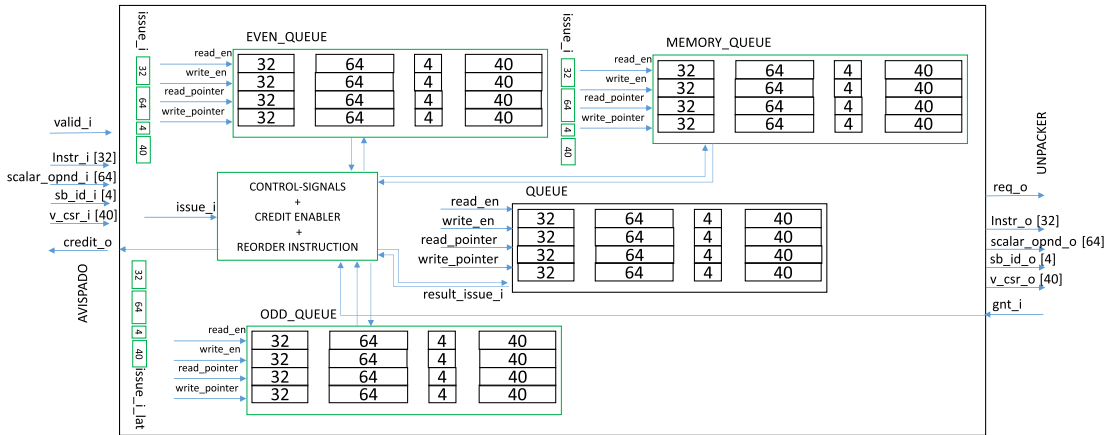


FIGURE 4. Fault tolerant pre-issue-queue with EVEN/ODD queue and separated memory queue.

(called EVEN and ODD) writes inside the ECC buffer the code word produced by the ECC logic. If the ECC comparison reports a mismatch, the instruction is not committed, and a kill / roll-back hardware procedure is activated to recover the status of the VPU as before the execution of the faulty instruction. The VPU units involved in vector instruction duplication and comparison are discussed below.

1) PRE-ISSUE QUEUE UNIT

The pre-issue queue is the first front-end unit of the VPU microarchitecture. It uses a credit-based system to get new instructions from OVI and send them to the Unpacker for decoding. In the original non-fault-tolerant version of the VPU, the unit releases a credit to the scalar core every time the VPU issues an instruction, notifying how many instructions the pipeline can accept [9]. In the FT version, the credits are released according to instruction duplication mechanism, counting one credit when issuing two replicas of an arithmetic instruction and one credit when issuing one memory instruction.

The pre-issue queue unit comprises three queues, as depicted in Fig. 4. The EVEN and ODD queues contain the duplicated arithmetic instructions, while the Memory queue contains the non-duplicated memory instructions. The control-credit-reorder block duplicates and splits the arithmetic instructions for the EVEN/ODD queues, also assigning them the EVEN/ODD label. The same block routes the memory instructions (assigned with memory label) to the memory queue. The duplicated instructions must be executed one after the other, yet without losing the commit order with respect to the memory instructions, which is guaranteed by the VROB unit.

2) UNPACKER UNIT

The unpacker block classifies the instructions, performing fine-grain decoding according to each instruction type, producing a set of control bits used by the lanes. Since instruction duplication is structured in the previous stage, this unit does not have any particular support for FT, just decoding the instructions one by one. The only added support for hardware instruction duplication is related to the vector slide

instructions, in which an additional vector move operation is inserted, compared to the original non-fault-tolerant version of the VPU, as detailed in Section IV-B5.

3) VECTOR REORDER BUFFER (VROB) UNIT

Differently from the original non-fault-tolerant version of the VPU, in the proposed microarchitecture the commit process of an arithmetic instruction cannot start until both EVEN and ODD copies of the same instruction are executed, and their results are compared in terms of ECC checking. The instruction can only be committed if both copies produce results with the same ECC signature. For this reason, we added the False Retire Manager hardware block to collect the Commits information from both EVEN and ODD instructions (Fig. 6). Once the EVEN instruction is completed, the block buffers all its committed information that can be related to a single instruction (or a group of micro instructions as it happens for widening / narrowing instructions) and then produces a *false retire signal*, which allows moving the head pointer of the VROB pointing to the ODD instruction. Once the ODD instruction is completed, the *completed* signal is sent to the scalar core, and the subsequent *retire_valid* signal causes the unit to check the *HW_Fault* signal coming from the lanes and reporting the outcome of the EVEN/ODD ECC comparison. If the EVEN/ODD ECC comparison does not detect any mismatch, the commit process can start. In case of *HW_Fault* assertion, the VROB activates a roll-back mechanism similar to the one triggered by an instruction kill command, leveraging the register renaming feature that allow to discard the vector being written. In the latest version of the design, the latency for roll-back sequence to recover the RAT and VROB configuration is as short as 3 clock cycles, thus leading to a very efficient fault recovery mechanism which minimizes the recovery overhead.

4) RENAMING UNIT

The Renaming Unit has been modified to support the execution of the duplicated Multiply - Accumulate (MAC) instructions. Without the proper support in the duplicated

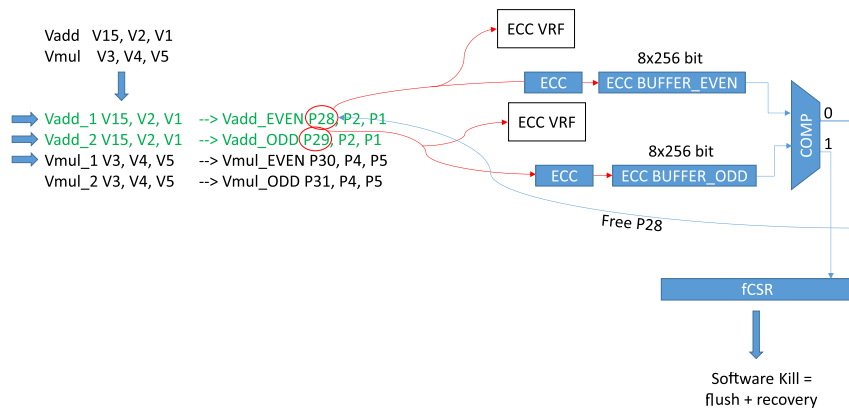


FIGURE 5. ECC buffer working principle for each Vector Lane. The duplicated instructions (Vadd in the example) are renamed with the physical registers P28 and P29 as destination registers. Each of the two instructions writes its result in the VRF and the ECC signature in the ECC Buffers for local comparison.

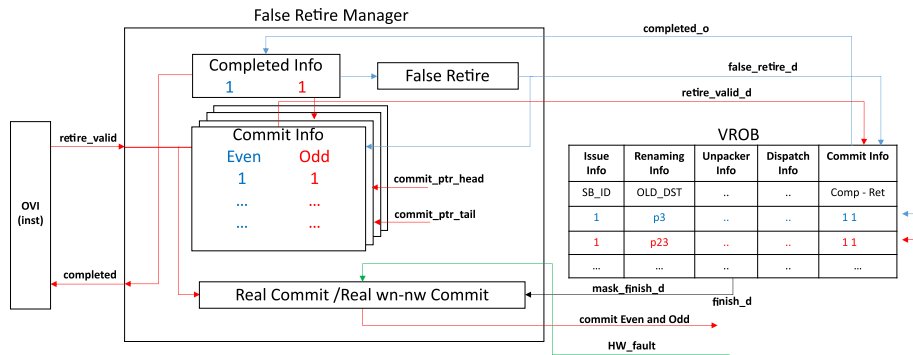


FIGURE 6. Fault tolerant VROB, with false retire manager block.

version of these instructions, the ODD instruction would accumulate the result of the MAC operation on the same register used by the EVEN instruction. To overcome this problem, the unit has been modified in such a way as to use the old destination register during the renaming of the ODD instruction.

5) SINGLE LANE UNIT

As previously explained, the VRF slice in each lane contains the ECC encoding and decoding mechanism used to check the correctness of the instructions, and so the comparison logic of the results is located in the VRF slice of each lane. When a duplicated instruction is executed, as occurs for the Vadd depicted in Fig. 5, the 8-bit ECC signatures calculated on the results produced by both instruction replicas (EVEN and ODD) are saved inside an ECC buffer. The ECC buffer in the lane occupy a total of $32 \times 8 \times 2$ bits since each lane accommodates 32 64-bit data elements for one vector, with two ECC buffers for the EVEN and ODD statements, respectively. The two buffers are compared after the ODD instruction is executed.

All the signals produced by the comparison in each lane are OR-combined and, in the case of a logic high outcome, a bit is set inside a custom CSR, blocking the VPU pipeline and in particular the commit process. If the comparison between

the ECC buffers has not detected any mismatch, the commit process continues, freeing the register in the VRF containing the result of the EVEN instruction (Physical register P28 in Fig. 5), as it has proven to be an exact duplication of the destination register of the ODD instruction. With this procedure, the instruction commit frees one copy of the produced result, not creating limitations in terms of VRF occupancy.

In the proposed design, we decided to avoid memory instruction duplication since it would be very complex to request a duplicated execution for load instructions, and it would be useless to execute a store instruction twice. To deal with this issue, the most suitable approach to protect the execution of memory instructions, particularly related to the execution of loads, is to replicate the load buffers individually with a TMR technique. The protection measures for memory operations may need to be improved, yet it is out of the scope of the present study and it is left to future improvements. Therefore, no ECC buffer checking and writing is done during memory instructions. However, the result produced by load operations is ECC-encoded and correctly written in the VRF.

A design feature of the Vitruvius+ microarchitecture is related to executing specific instructions, such as narrowing and widening arithmetic operations, by splitting them into micro-operations directly in hardware during the Unpacker phase. In this case, only the ECC buffers produced by the

last valid micro-operation are compared when executing arithmetic instructions containing micro-operations.

Among the arithmetic instructions, a specific corner case is related to vector slide operations. In the original non-fault-tolerant version of Vitruvius+, vector slide operations are translated with two micro-operations: a vector move and a vector slide. Since the slide operations are also applicable with Single Element Width (SEW) values other than 64 bits (e.g. 32,16,8), the ECC mechanism could generate incorrect values as this can only encode 64-bit values. To overcome this problem, during these instructions, vector move and vector slide operations are performed without ECC-encoding, and a final vector move operation is added with the purpose of reading the result register of the vector slide and writing it back into the VRF with ECC-encoding. The only limitation of this approach is the lack of ECC protection within the result register during vector sliding.

V. EXPERIMENTAL RESULTS

Incorporating an ECC support into the VRF necessitates a comprehensive modification of the read/write mechanism. This involves reorganizing the data, adding encoding and decoding logic blocks, and re-synthesizing a larger register file to accommodate the parity bits appended to each data word. The consequent impact on the design and verification time can be quantified in one month's work, leaving apart the addition of instruction duplication hardware management that is also part of our work. From a verification standpoint, the UVM process was adapted to include fault injection procedures and the associated controls, which were essential for validating the integration of ECC logic. In this section, we present the area and speed comparison between the proposed design and the original VPU, and the experimental results obtained by an extensive fault injection campaign on the entire VPU.

A. IMPACT ON HARDWARE RESOURCES

In order to evaluate the silicon area and timing overhead introduced by the new features of the VPU design, the entire VPU was synthesized for the GF22FDX technology using Cadence Genus Synthesis Solution 19.11, with exactly the same synthesis constraints and options for both the original Vitruvius+ design and the proposed fault-tolerant Vitruvius+ design. We selected the 8-Tracks (8T) standard cell library, first implementing the 72-bit register to implement the VRF Static RAM block. In the synthesis process, we enabled the automatic retiming feature and the automatic clock gating feature, setting the target frequency at 1.25 GHz (800 ps cycle). We set the synthesizer for using both Super Low Vt (SLVT) and Low Vt (LVT) cells, the former in the timing critical paths parts and the latter in the elements with no timing issues [9].

As depicted in Table 1, in the typical corner case (TT / 0.80 V / 25 °C), the proposed fault-tolerant design successfully satisfies the timing constraints, achieving an estimated maximum frequency of 1.47 GHz with the critical path set in the Register File due to the newly introduced ECC mechanisms, compared to 1.51 GHz reached from the original

TABLE 1. Synthesis Results Using Cadence Genus Synthesis Solution 19.11 on the 8T GF22FDX Library

	VPU Area	VPU Frequency (typ)	VPU Frequency (slow)
No ECC - No HW-ID	1.49 μm^2	1.51 GHz	1.08 GHz
ECC - HW-ID	1.61 μm^2	1.47 GHz	1.04 GHz

Vitruvius+ design. The maximum frequency in the slow corner case (SS / 0.72 V / 125 °C) is around 1.04 GHz, compared to 1.08 GHz in the original Vitruvius+ design.

Table 1 also reports the area occupation of the entire VPUs. By comparing the proposed fault tolerant design with the Vitruvius+ design, we obtained a hardware overhead of 7.5%, of which 6.83% is due to the architectural changes made for the implementation of ECC in the Register File and 0.67% is due to the hardware support for instruction duplication. In the overall area breakdown, the single lane represents the most area-consuming component, having the FPU and the VRF as larger units. As an outcome, only 10% of the entire VPU signal distribution is occupied by non-lane blocks. This aspect is relevant in the view of the fault injection simulation campaign, as the probability that a faulty bit occurs in the lanes is circa ten times higher than the probability that it occurs in the non-lane area. Further details on the synthesis processes and power consumption for the original Vitruvius+ can be found in [9].

B. VERIFICATION METHODOLOGY

The original design of the Vitruvius+ VPU underwent functional verification, carried out on the QuestaSim-64 2021.3_2 RTL simulator. The verification relies on the simulated execution of randomly generated binaries as well as directed test programs, within a dedicated Universal Verification Methodology (UVM) environment that employs a co-simulation approach, integrating the Spike RISC-V ISA simulator as the golden reference. The UVM verification environment originally consists of two main parts [38], [39]:

- A general UVM environment, shared across multiple projects, that can generate and simulate vector instructions on Spike and on the Device Under Test (DUT), including a UVM scoreboard module that compares the output of Spike and of the DUT for each instruction;
- A Continuous Integration (CI) framework featuring sanity checks, random test generation, and coverage collection.

Since the direct verification is implemented on per-instruction synchronization basis with Spike inside the UVM environment, any mismatch found by the scoreboard module results in a debug information prompt, easing the debugging of the architecture. In order to carry out fault injection simulations injecting Single Event Upset (SEU), the UVM environment was enriched with a bit flip mechanism [40] via the *uvm_hdl_deposit* function, monitoring the *HW_Fault* signal that comes out of the lanes to notify a hardware fault detection to the False Retired Manager unit, as explained in Section IV-B3. As a fault simulation approach, we adopted a cumulative FI analysis, which, as opposed to random Monte

TABLE 2. Fault Test Setup for ISA Benchmarks

	Duration [ps]	N_Fault
Single_Width_Integer_Multiply	237285	79
Integer_Divide	717405	239
Widening_Integer_MultiplyAdd	194525	65
Single_Width_Integer_AddSubtract	32045	11
Narrowing_Integer_Right_Shift	127845	43
Single_Width_Averaging_Add_Subtract	34845	12
Single_Width_Floating_Point_Integer_Convert	12005	4
Single_Width_Floating_PointMultiplyDivide	200785	67
Widening_Floating_Point_Fused_MultiplyAdd	73045	24
Narrowing_Floating_Point_Integer_Convert	27825	9

Carlo FI, deterministically injects faults at very narrow intervals in each of the sensitive bits of the microarchitecture, by means of repeated simulation runs. The approach allows characterizing the fault detection coverage in a much more accurate way than random FI.

The fault analysis was conducted on the execution of specific ISA benchmark tests to independently activate specific parts of the hardware. The implemented setup is shown in Table 2 and presents the various tests along with the duration and fault quantity per single run. The target bits of injected faults were chosen taking into account the synthesis results described in Section V-A, from which it can be seen that the majority of the VPU signals (circa 90%) are distributed within the vector lanes. Moreover, given the repetitive structure of the 8 identical interconnected lanes, fault injection simulations were chosen to be implemented only on signals belonging to Lane 0, considering the obtained results extendable to all the lanes. Notably, the inter-lane interconnect allows data movement among the lanes – as it happens in the execution of vector slicing instructions, and therefore the possibility of error propagation among the lanes is also taken into consideration. To facilitate carrying out the FI simulations, also reducing their duration, it was chosen to analyze only the 0-index bit of all the signals belonging to Lane 0, for a total of approximately 3400 faulty bits per simulation run. The UVM environment reads the list of signals and automatically performs FI simulation on each of them, according to the fault setup depicted in 2. Given the different test durations and the different latencies that vector instructions may have, the fault rate of the cumulative FI tests was set to an average of 1 fault every 300 clock cycles to avoid multiple faults hitting two copies (EVEN and ODD) of the same instruction, misleading the detection system. Notably, the occurrence of two consecutive faults on the same bit of the same lane within less than 300 clock cycles would be a statistically non-significant scenario for the real operation of the system.

The UVM FI system can collect four different types of information by means of the comparison with Spike, described as follows:

- **SUCCESS:** the comparison of the results created by each instruction did not produce any errors despite the injection of a fault, akin to a masked silent error in the FT literature parlance;
- **HW_FAULT_DETECTED:** the VPU detects the fault through the hardware fault detection system

TABLE 3. Fault Test Setup for ISA Benchmarks - Unprotected VPU

	Duration [ps]	N_Fault
Single_Width_Integer_Multiply	119585	39
Integer_Divide	358945	120
Widening_Integer_MultiplyAdd	97145	32
Single_Width_Integer_AddSubtract	16265	5
Narrowing_Integer_Right_Shift	63945	21
Single_Width_Averaging_Add_Subtract	17665	6
Single_Width_Floating_Point_Integer_Convert	6205	2
Single_Width_Floating_PointMultiplyDivide	100605	34
Widening_Floating_Point_Fused_MultiplyAdd	36505	12
Narrowing_Floating_Point_Integer_Convert	14025	5

implemented and described in the previous Sections. In this case, the fault is managed and resolved either by hardware roll-back or by VRF scrubbing, depending on the origin of the fault. This corresponds to a detected and corrected error;

- **TIMEOUT/HANG:** the VPU reaches a stall state due to an error, no longer being able to commit any instructions. This corresponds to a hang error at the application level.
- **SB_MISMATCH:** the scoreboard module notifies a mismatch between the results produced by the RTL design simulations and the Spike simulation. This corresponds to an escaped fault.

Both the TIMEOUT/HANG and the SB_MISMATCH conditions highlight that a random fault has not been detected by the implemented hardware fault tolerant features and has produced a program failure.

Fig. 7 depicts the implemented fault injection analysis results. These graphs reveal a variation in outcomes based on the type of benchmark analyzed, with a notable occurrence of success rates exceeding 75% in many benchmarks, thus evidencing that most of the examined bits do not represent ACE bits. Regarding the observed errors, the hardware fault detection mechanism worked more than 60% of the time, proving the functioning of the implemented technique. As it is common in microprocessor systems subject to the occurrence SEUs, TIMEOUT/HANG type errors constitute a significant portion of the application failures [41]. Such errors manifest when a fault impacts a bit involved in architecture control sequences, such as control signals or FSM signals. These errors may be addressed through the implementation of watchdog timer subsystems along with conventional checkpoint/rollback failure recovery, which, however, are out of the scope of this work. SB_MISMATCH-type errors represent an interesting case study since they escape the control of the hardware fault detection system. It can be reasonably assumed they are due to faults affecting cache memory access operations, which are not addressed by the proposed design and may be overridden by means of conventional ECC-based cache memory protection. On average, referring to Fig. 7, HW_FAULT_DETECTED outcomes count for 14% of the total fault injection tests (60% of non-silent faults), TIMEOUT/HANG outcomes count for 8.7% of the total (30% of non-silent faults), and SB_MISMATCH outcomes count for 3.7% of the total (10% of non-silent faults). In order to

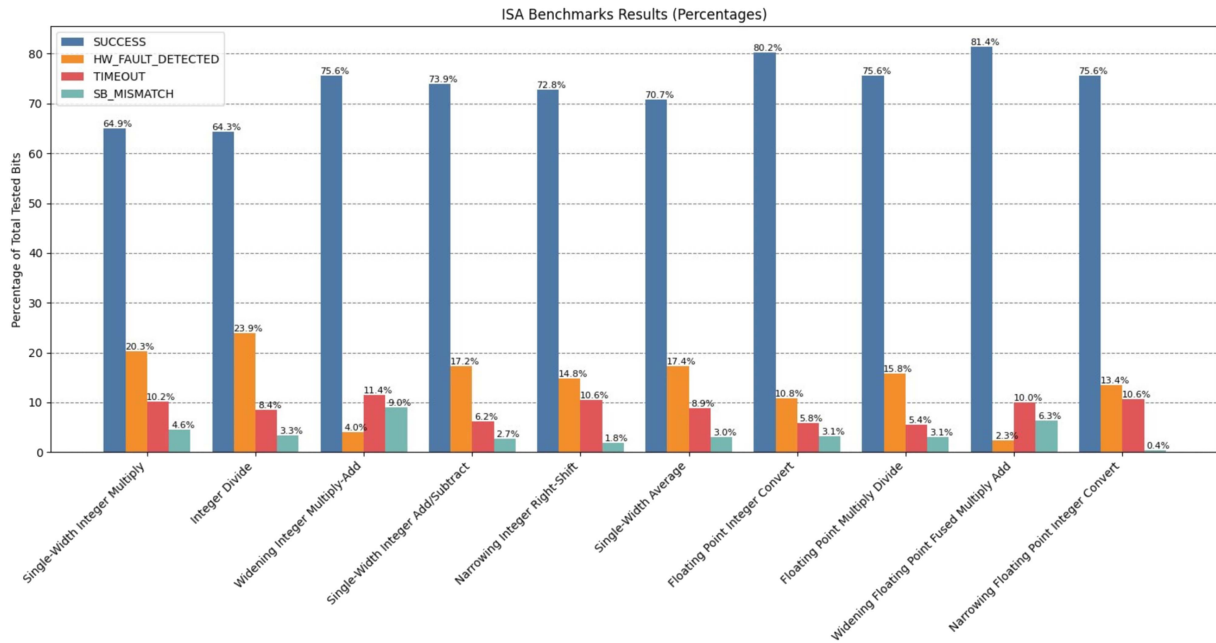


FIGURE 7. Fault injection results on RISCv rvv-1.0 ISA benchmarks.

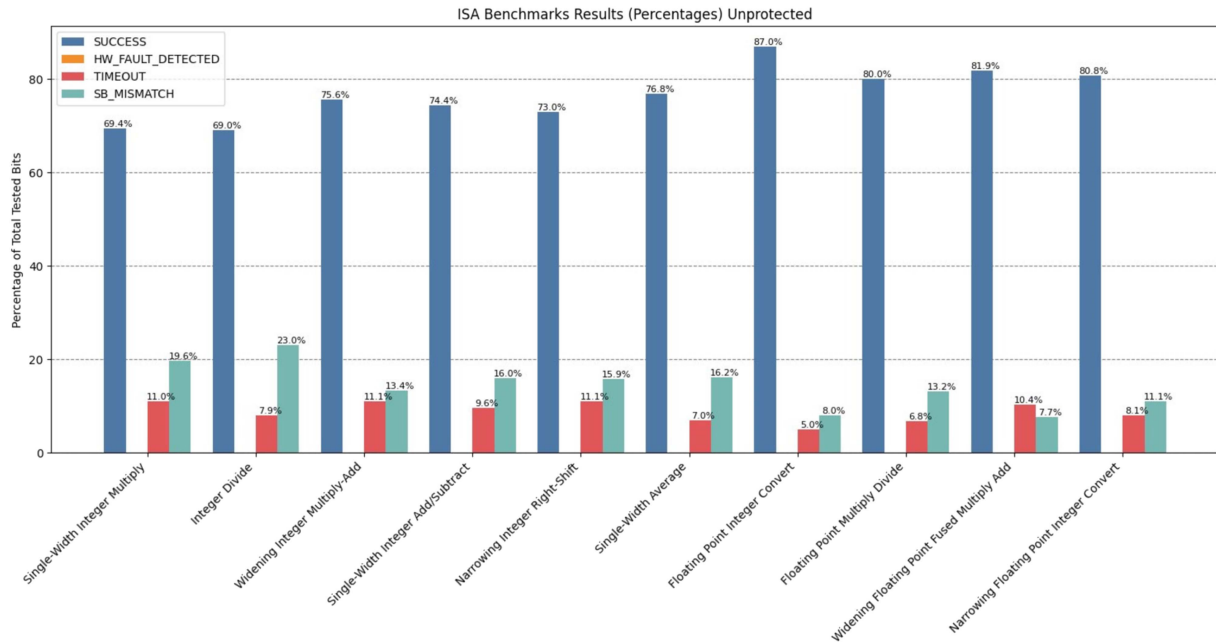


FIGURE 8. Fault injection results on RISCv rvv-1.0 ISA benchmarks for the unprotected architecture.

measure the effectiveness of the approach, we performed the same cumulative FI analysis on the original non-redundant design of the VPU. Fig. 8 depicts the FI results.

Despite the lower quantity of faults inserted, which are reported in Table 3 and are due to the non-duplication of the instructions and the consequent shorter test duration, the TIMEOUT failure percentages remained largely consistent with those of the Fault-Tolerant (FT) version, while the SB_MISMATCH failure percentages noticeably increased. The SUCCESS percentage, reflecting silent faults, exhibits a

slight increase, particularly in the tests with fewer injected faults. On average, the TIMEOUT/HANG outcomes, which are preventable by traditional watchdog counter mechanism, count for 8.8% of the total injected faults in the original VPU design, while they are 8.7% in the proposed design. SB_MISMATCH outcomes, which correspond to escaped faults, count for 14.4% of the total in the original VPU design while they are 3.7% in the proposed design, thus obtaining a 75% reduction in escaped faults causing application failure.

VI. CONCLUSION

The proposed work reports the design and the validation of a fault tolerant VPU for HPC systems. The work leverages the Vitruvius+ architecture developed within the European Processor Initiative project, extending it with vector instruction-level Temporal Redundancy and Error Correcting Codes. Specifically, we described all the employed hardware modifications inside the main sensitive architectural units such as the Lanes, Pre-Issue Queue, Unpacker, VROB and Renaming Unit, implementing the Hardware Instruction Duplication technique within the decoding stages of the VPU architecture, along with an ECC-based protection for the VRF that is also used as a signature for duplicated instruction result comparison. The design complements traditional software-based checkpoint/roll-back techniques, with only 7% area overhead calculated with respect to the sole VPU area, and limiting the performance impact to the minimum inherent overhead of instruction duplication, thanks to the purely hardware implementation. A general observable impact on computing speed is related to the slight decrease in the working frequency of the processor. However, like in any fault-resilience redundancy mechanism, the computation overhead associated to instruction duplication can actually reduce the overall performance for both latency-sensitive and throughput-sensitive workloads. In the worst-case scenario, where all instructions are arithmetic, this overhead can reach 2X in the execution time. However, this value does not change in a multi-node environment, differently from the worst-case of 6X-8X time overhead obtained with coordinated/uncoordinated checkpointing [5], the 2.8X obtained with execution/verify approaches and the 4X given by parallelized approaches with checkpointing [20], [21]. The trade-off is often acceptable when considering the enhanced reliability and fault tolerance provided under certain operational conditions requiring a high safety level. Additionally, disabling the instruction duplication mechanism allows one to restore the system full speed when necessary. We both conducted a Synthesis evaluation to prove the hardware impact of the proposed approach and performed RTL fault-injection tests based on different benchmark programs. The introduced features reduce by 75% the occurrence of non-silent undetected faults that would result in application failure. Generally speaking, the proposed method demonstrates reliability across all operational modes, including long-term operation, concerning the tested SEUs. However, challenges related to Total Ionizing Dose (TID) and similar issues arising from environmental conditions cannot be addressed solely through design strategies. Instead, these concerns should be mitigated through appropriate anti-ageing technology choices.

REFERENCES

- [1] R. D. Sala and G. Scotti, "A novel FPGA implementation of the NAND-PUF with minimal resource usage and high reliability," *Cryptography*, vol. 7, no. 2, 2023, Art. no. 18. [Online]. Available: <https://www.mdpi.com/2410-387X/7/2/18>
- [2] J. Dongarra, T. Herault, and Y. Robert, *Fault Tolerance Techniques for High-Performance Computing*. Berlin, Germany: Springer, 2015.
- [3] F. Cappello, "Fault tolerance in petascale/exascale systems: Current knowledge, challenges and research opportunities," *Int. J. High Perform. Comput. Appl.*, vol. 23, no. 3, pp. 212–226, 2009.
- [4] B. Schroeder and G. A. Gibson, "Understanding failures in petascale computers," *J. Phys. Conf. Ser.*, vol. 78, no. 1, 2007, Art. no. 012022.
- [5] O. M. Mendizabal, F. L. Doti, and F. Pedone, "Analysis of checkpointing overhead in parallel state machine replication," in *Proc. 31st Annu. ACM Symp. Appl. Comput.*, New York, NY, USA, 2016, pp. 534–537, doi: [10.1145/2851613.2851879](https://doi.org/10.1145/2851613.2851879).
- [6] R. Espasa, M. Valero, and J. Smith, "Vector architectures: Past, present and future," in *Proc. Int. Conf. Supercomput.*, New York, NY, USA, 1998, pp. 425–432, doi: [10.1145/277830.277935](https://doi.org/10.1145/277830.277935).
- [7] R. Egawa et al., "Potential of a modern vector supercomputer for practical applications: Performance evaluation of SX-ACE," *J. Supercomput.*, vol. 73, pp. 3948–3976, 2017.
- [8] M. Kovač, "European processor initiative: The industrial cornerstone of EuroHPC for exascale era," in *Proc. 16th ACM Int. Conf. Comput. Front.*, New York, NY, USA, 2019, Art. no. 319, doi: [10.1145/3310273.3323432](https://doi.org/10.1145/3310273.3323432).
- [9] F. Minervini et al., "Vitruvius : An area-efficient risc-v decoupled vector coprocessor for high performance computing applications," *ACM Trans. Archit. Code Optim.*, vol. 20, no. 2, pp. 1–25, 2023.
- [10] C. Lomont, "Introduction to intel advanced vector extensions," *Intel White Paper*, vol. 23, no. 23, pp. 1–21, 2011.
- [11] N. Stephens et al., "The arm scalable vector extension," *IEEE Micro*, vol. 37, no. 2, pp. 26–39, Mar./Apr. 2017.
- [12] J. K. Lee, M. Jamieson, N. Brown, and R. Jesus, "Test-driving RISC-V vector hardware for HPC," in *Proc. High Performance Comput.*, 2023, pp. 419–432.
- [13] P. Vizcaino, F. Mantovani, R. Ferrer, and J. Labarta, "Acceleration with long vector architectures: Implementation and evaluation of the FFT kernel on NEC SX-aurora and RISC-V vector extension," *Concurrency Comput. Pract. Experience*, vol. 35, no. 20, 2023, Art. no. e7424.
- [14] M. Sato, "The supercomputer "Fugaku" and arm-SVE enabled A64FX processor for energy-efficiency and sustained application performance," in *2020 19th Int. Symp. Parallel Distrib. Comput.*, 2020, pp. 1–5.
- [15] O. Subasi, F. Zylkyarov, O. Unsal, and J. Labarta, "Marriage between coordinated and uncoordinated checkpointing for the exascale era," in *2015 IEEE 17th Int. Conf. High Perform. Comput. Commun.*, 2015 IEEE 7th Int. Symp. Cyberspace Saf. Secur., 2015 IEEE 12th Int. Conf. Embedded Softw. Syst., 2015, pp. 470–478.
- [16] A. Moody, G. Bronevetsky, K. Mohror, and B. R. De Supinski, "Design, modeling, and evaluation of a scalable multi-level checkpointing system," in *Proc. 2010 ACM/IEEE Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2010, pp. 1–11.
- [17] P. H. Hargrove and J. C. Duell, "Berkeley lab checkpoint/restart (BLCR) for Linux clusters," *J. Phys., Conf. Ser.*, vol. 46, no. 1, 2006, Art. no. 494.
- [18] G. Kingsley, M. Beck, and J. S. Plank, "Compiler-assisted checkpoint optimization using SUIF," in *Proc. 1st SUIF Compiler Workshop*, Cite-seer, 1995, pp. 1–16.
- [19] L. Bautista-Gomez et al., "FTI: High performance fault tolerance interface for hybrid systems," in *Proc. 2011 Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2011, pp. 1–32.
- [20] R. Kotla and M. Dahlin, "High throughput byzantine fault tolerance," in *Proc. IEEE Int. Conf. Dependable Syst. Netw.*, 2004, pp. 575–584.
- [21] M. Kapritsos, Y. Wang, V. Quema, A. Clement, L. Alvisi, and M. Dahlin, "All about eve: {execute-verify} replication for {multi-core} servers," in *Proc. 10th USENIX Symp. Operating Syst. Des. Implementation*, 2012, pp. 237–250.
- [22] B. Munger et al., "'Zen 4': The AMD 5nm 5.7 GHz x86-64 micro-processor core," in *2023 IEEE Int. Solid-State Circuits Conf.*, 2023, pp. 38–39.
- [23] W. Chen et al., "A 22nm 2.5MB slice on-die L3 cache for the next generation xeon processor," in *2013 Symp. VLSI Circuits*, 2013, pp. C132–C133.
- [24] L. Osinski, T. Langer, and J. Mottok, "A survey of fault tolerance approaches on different architecture levels," in *Proc. 30th Int. Conf. Archit. Comput. Syst.*, 2017, pp. 1–9.
- [25] A. Aponte-Moreno, F. Restrepo-Calle, and C. Pedraza, "A low-cost fault tolerance method for arm and RISC-V microprocessor-based systems using temporal redundancy and approximate computing through simplified iterations," *J. Integr. Circuits Syst.*, vol. 16, no. 3, pp. 1–14, 2021.

- [26] M. Goma, C. Scarbrough, T. Vijaykumar, and I. Pomeranz, "Transient-fault recovery for chip multiprocessors," *ACM SIGARCH Comput. Archit. News*, vol. 31, no. 2, pp. 98–109, 2003.
- [27] E. Rotenberg, "Ar-smt: A microarchitectural approach to fault tolerance in microprocessors," in *Proc. IEEE 29th Annu. Int. Symp. Fault-Tolerant Comput.*, 1999, pp. 84–91.
- [28] M. Barbirotta, F. Menichelli, A. Cheikh, A. Mastrandrea, M. Angioli, and M. Olivieri, "Dynamic triple modular redundancy in interleaved hardware threads: An alternative solution to lockstep multi-cores for fault-tolerant systems," *IEEE Access*, vol. 12, pp. 95720–95735, 2024.
- [29] M. Barbirotta, A. Cheikh, A. Mastrandrea, F. Menichelli, M. Ottavi, and M. Olivieri, "Evaluation of dynamic triple modular redundancy in an interleaved-multi-threading RISC-V core," *J. Low Power Electron. Appl.*, vol. 13, 2022, Art. no. 2.
- [30] S. S. Mukherjee, M. Kontz, and S. K. Reinhardt, "Detailed design and evaluation of redundant multithreading alternatives," *ACM SIGARCH Comput. Archit. News*, vol. 30, no. 2, pp. 99–110, 2002.
- [31] S. Arslan and O. Ünsal, "Efficient thread-to-core mapping alternatives for application-level redundant multithreading," *Concurrency Comput. Pract. Experience*, vol. 35, 2023, Art. no. e7622.
- [32] S. Alcaide, L. Kosmidis, C. Hernandez, and J. Abella, "Software-only based diverse redundancy for ASIL-D automotive applications on embedded HPC platforms," in *2020 IEEE Int. Symp. Defect Fault Tolerance VLSI Nanotechnol. Syst.*, 2020, pp. 1–4.
- [33] R. Hyman, K. Bhattacharya, and N. Ranganathan, "Redundancy mining for soft error detection in multicore processors," *IEEE Trans. Comput.*, vol. 60, no. 8, pp. 1114–1125, Aug. 2011.
- [34] E. Mizan, T. Amimeur, and M. F. Jacome, "Self-imposed temporal redundancy: An efficient technique to enhance the reliability of pipelined functional units," in *Proc. IEEE 19th Int. Symp. Comput. Archit. High Perform. Comput.*, 2007, pp. 45–53.
- [35] M. Barbirotta et al., "Fault-tolerant hardware acceleration for high-performance edge-computing nodes," *Electronics*, vol. 12, no. 17, 2023, Art. no. 3574. [Online]. Available: <https://www.mdpi.com/2079-9292/12/17/3574>
- [36] RISC-V vector extension version 1.0, 2023. [Online]. Available: <https://github.com/riscv/riscv-v-spec/blob/v1.0/v-spec.adoc>
- [37] The open vector interface, 2023. [Online]. Available: <https://github.com/semidynamics/OpenVectorInterface>
- [38] J. Quiroga et al., "Reusable verification environment for a RISC-V vector accelerator," in *Proc. Des. Verification Conf. Exhib. Europe*, Munich, pp. 6–7, 2023.
- [39] V. Jimenez et al., "Functional verification of a RISC-V vector accelerator," *IEEE Des. Test*, vol. 40, no. 3, pp. 36–44, Jun. 2023.
- [40] M. Barbirotta et al., "Fault resilience analysis of a RISC-V microprocessor design through a dedicated UVM environment," in *Proc. 33rd IEEE Int. Symp. Defect Fault Tolerance VLSI Nanotechnol. Syst.*, 2020, pp. 1–6.
- [41] Y. Zhu et al., "What is system hang and how to handle it," in *2012 IEEE 23rd Int. Symp. Softw. Rel. Eng.*, 2012, pp. 141–150.

Open Access funding provided by 'Università degli Studi di Roma "La Sapienza" 2' within the CRUI CARE Agreement